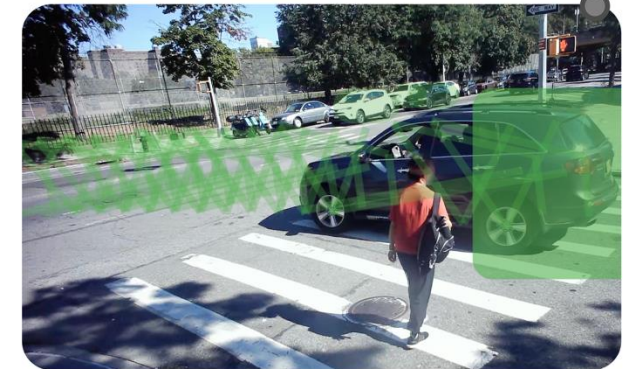
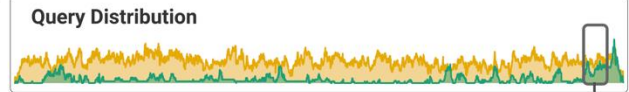
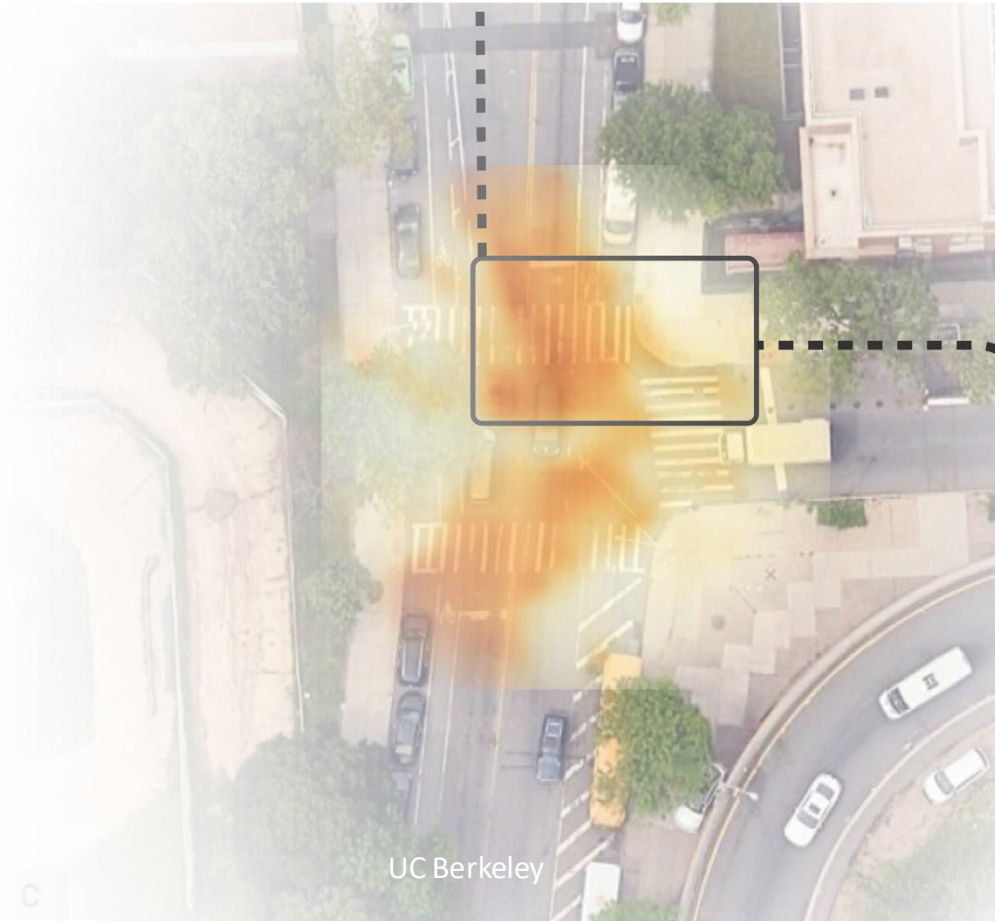
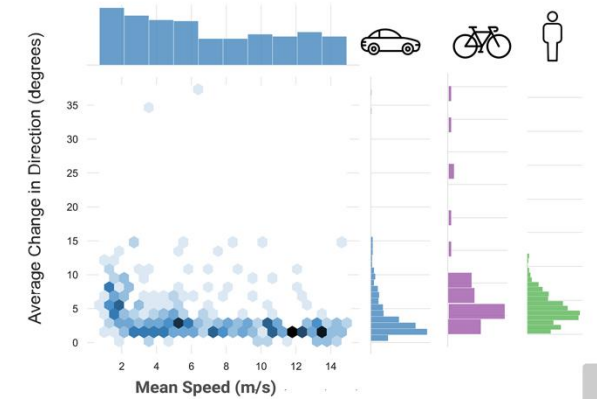
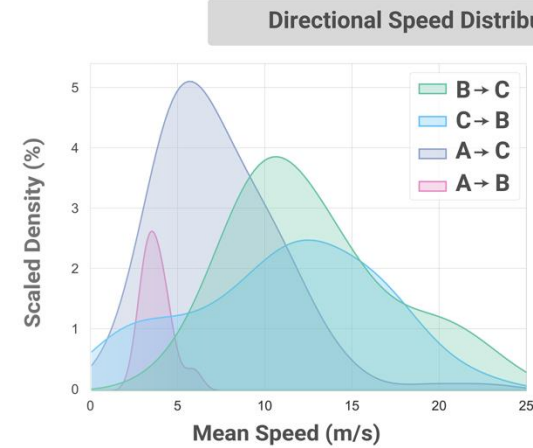
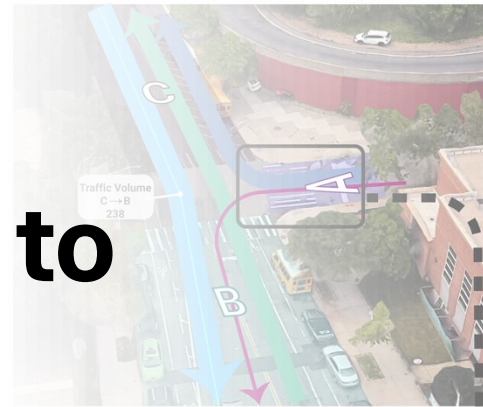


Introduction to Urban Data Analytics

Fall 2026

Maryam Hosseini

City & Regional Planning



UC Berkeley

C

B

D

Spatial Predicates

Spatial questions

Which parcels fall inside a historic district boundary?

How many bus stops are inside this neighborhood?

Which sidewalk segments intersect flood-prone areas?

Which intersections are within 500 ft of a crash hotspot?

How many POIs are within 1,000 ft of this site?

Which transit stop is nearest to this location?

How many grocery stores are within a 10-minute walkshed of this census block?

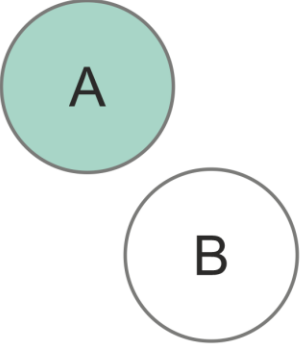
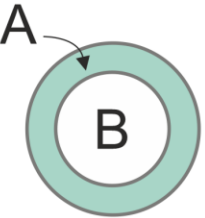
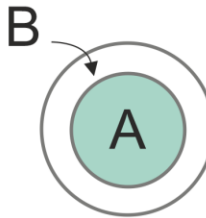
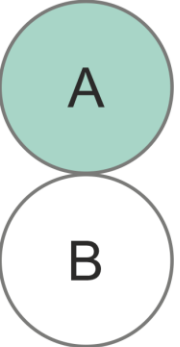
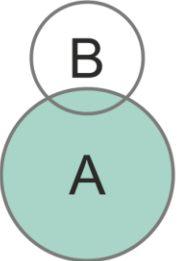
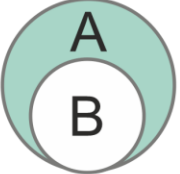
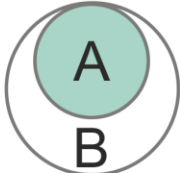
Which clinic is nearest by walking time from this housing site?

Spatial Questions, Spatial Methods

Example question	Output	Spatial relationship	Operation / predicate
Which restaurants fall inside this neighborhood boundary?	Select	Containment	within
Which sidewalk segments are located in flood-prone areas?	Select	Intersection	intersects
Which intersections are within 500 ft of a crash location?	Select	Distance threshold	dwithin
How many POIs are within 1,000 ft of each site?	Count	Distance threshold	dwithin + count
Which hospital is nearest to each senior housing building?	Match nearest	Nearest neighbor	Nearest search
How many residential locations are inside high heat-risk zones?	Count	Containment	within + count

Spatial Predicates

Contains excludes boundary-only containment

 disjoint <i>"A and B are fully disconnected"</i>	 contains <i>"A contains B"</i>	 within <i>"A is within B"</i>	 equals <i>"A and B are topologically equal"</i>
 touches <i>"A and B touch at border"</i>	 overlaps <i>"A and B overlap each other"</i>	 covers <i>"A covers B"</i>	 covered by <i>"A is covered by B"</i>

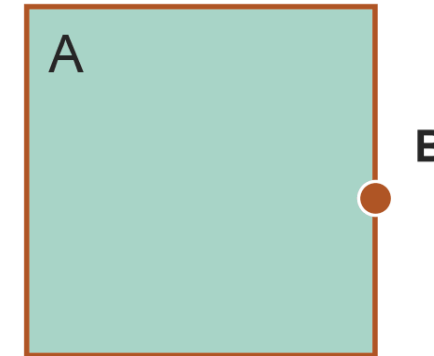
Spatial Relationships (Predicates)

Spatial predicate:
A yes/no spatial test
between two features.

- **Intersects (A, B)** → true if A and B share at least one point
- **Touches (A, B)** → true if A and B share a boundary but not their interior
- **Crosses (A, B)** → true if A *interior intersects* the B *interior* but does not contain it
- **Contains (A, B)** → True if **every point of B is in A, and** at least one interior point of B lies in the interior of A (so boundary-only cases can fail).
- **Covers (A, B)** → true if **every point of B is in A or on A's boundary** (boundary-only cases are allowed).
- **Within (A, B)** → true if A is completely inside B
- **Equals (A, B)** → true if A and B represent the same geometric shape
- **Disjoint (A, B)** → true if A and B have no points in common

Contains vs. Covers

- **Contains** excludes boundary-only containment. A polygon returns FALSE if a smaller geometry (a line or point) lies entirely on its boundary rather than intersecting its interior
- The DE-9IM intersection matrix requires the interiors of both geometries to share at least one point for **Contains** to be true
- **Covers** is more inclusive. It returns TRUE for boundary-only cases, since it only requires that no points of B lie in the exterior of A

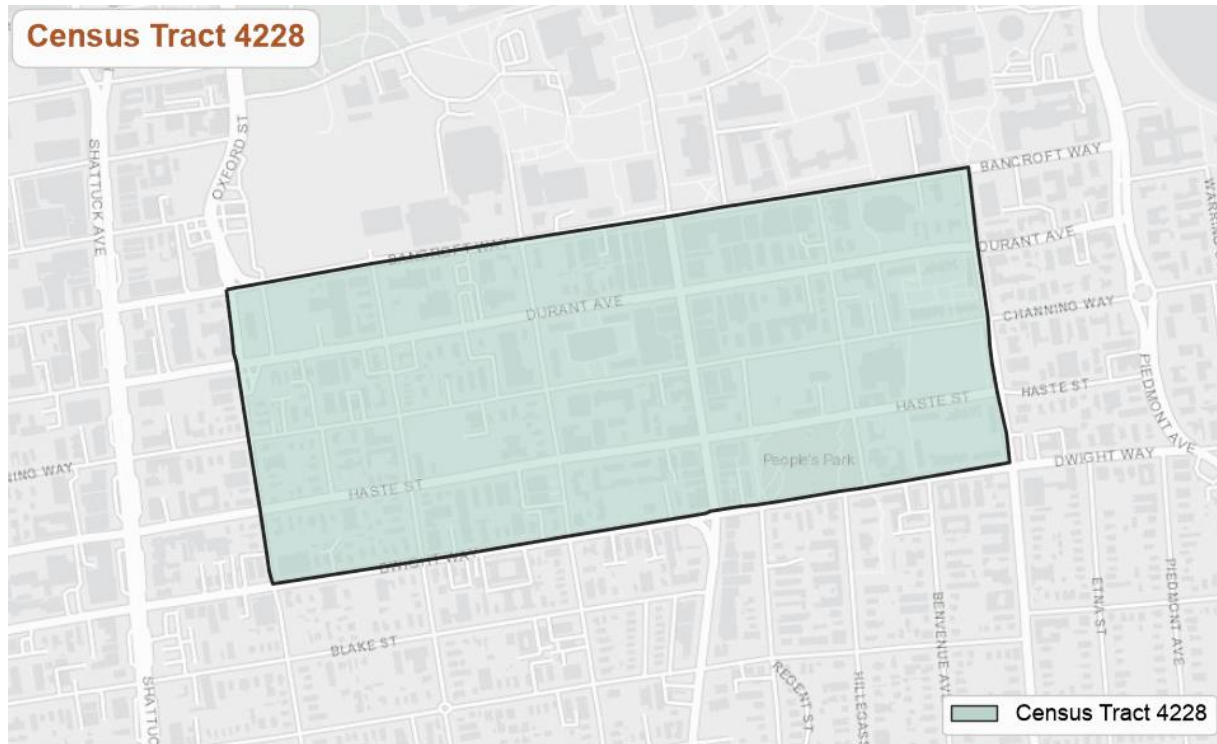


B (point) lies only on the boundary of A

A.contains(B) **False**

A.covers(B) **True**

Containment - within



```
tract = tracts[tracts["NAME"] == "4228"].geometry.iloc[0]
eat[eat.within(tract)]
```

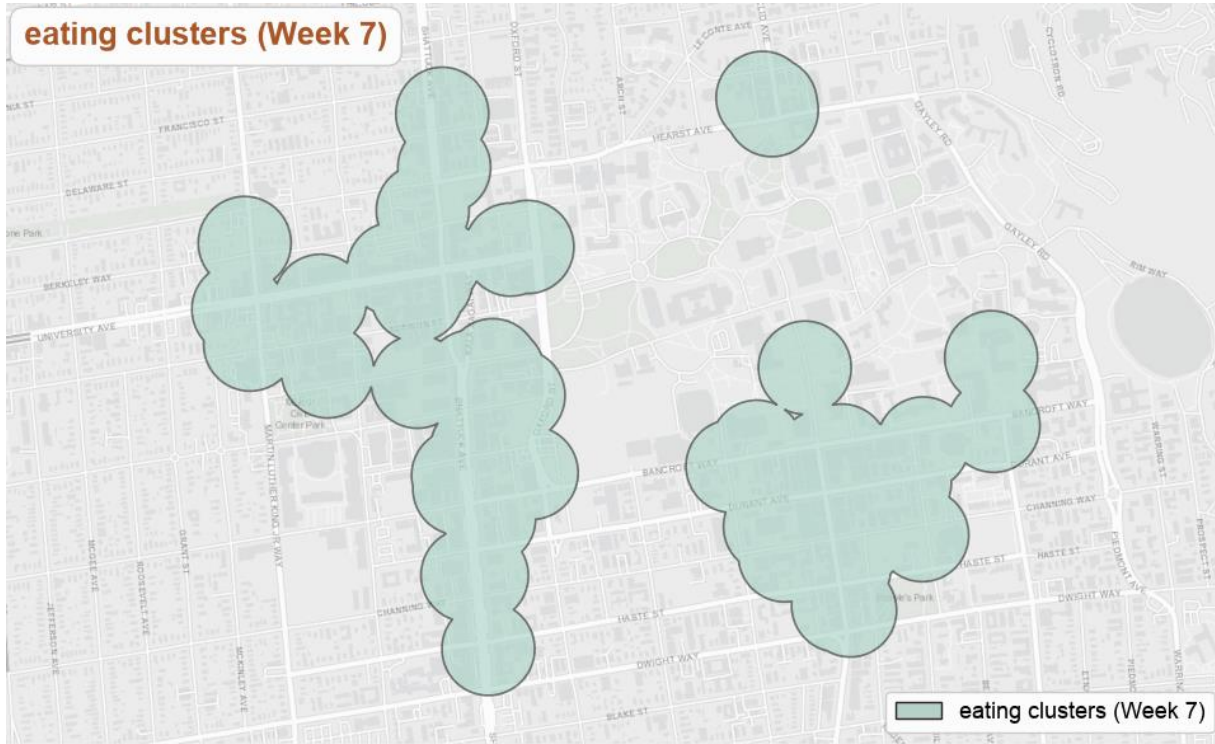
Which of the class's eating places fall inside Census Tract 4228?

Output	Select
Spatial relationship	Containment
Operation / predicate	within

- **within** returns one True or False per place
- 38 of 94 eating places are inside the tract
- A point exactly on the tract boundary returns False

Same method as
Which restaurants fall inside this neighborhood boundary?

Intersection - intersects



```
eat_clusters = clusters[clusters["cluster_type"] == "eat"]  
streets[streets.intersects(eat_clusters.union_all())]
```

Which street segments intersect the class's eating clusters?

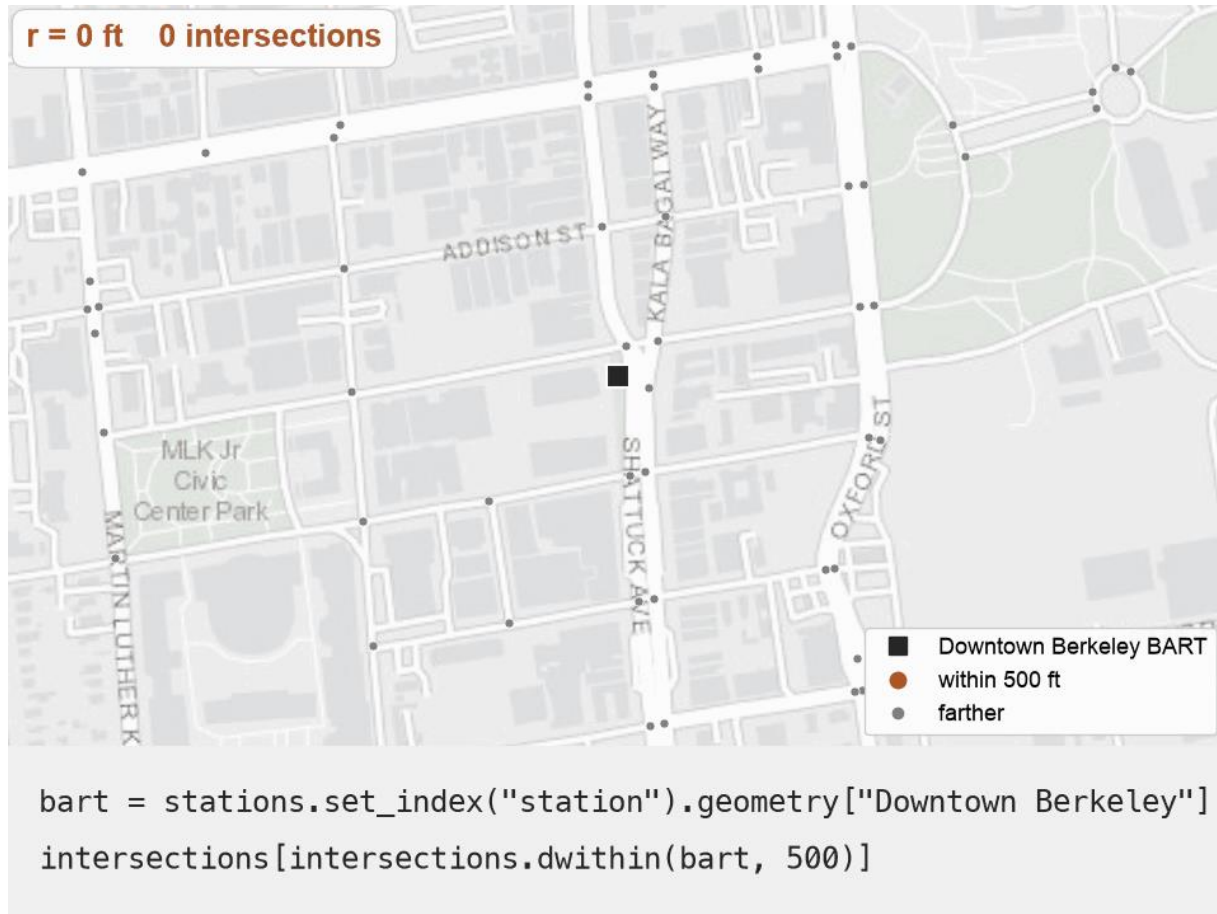
Output	Select
Spatial relationship	Intersection
Operation / predicate	intersects

- **intersects** is True if a segment shares any point with the clusters
- 275 segments intersect; only 157 lie fully within
- Segments that cross the cluster edge count for intersects, not for within

Same method as

Which sidewalk segments are located in flood-prone areas?

Distance Threshold - dwithin



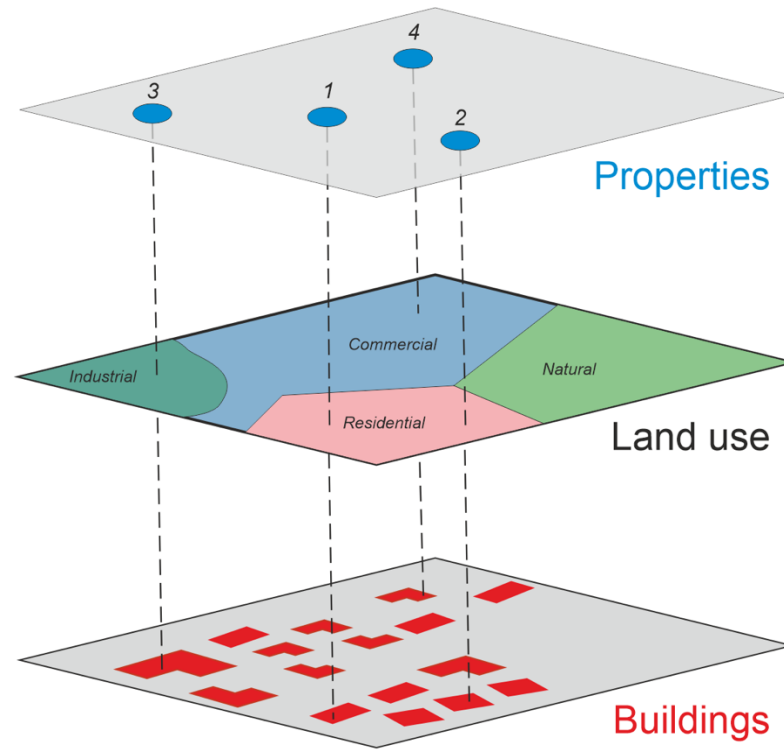
Which street intersections are within 500 ft of Downtown Berkeley BART?

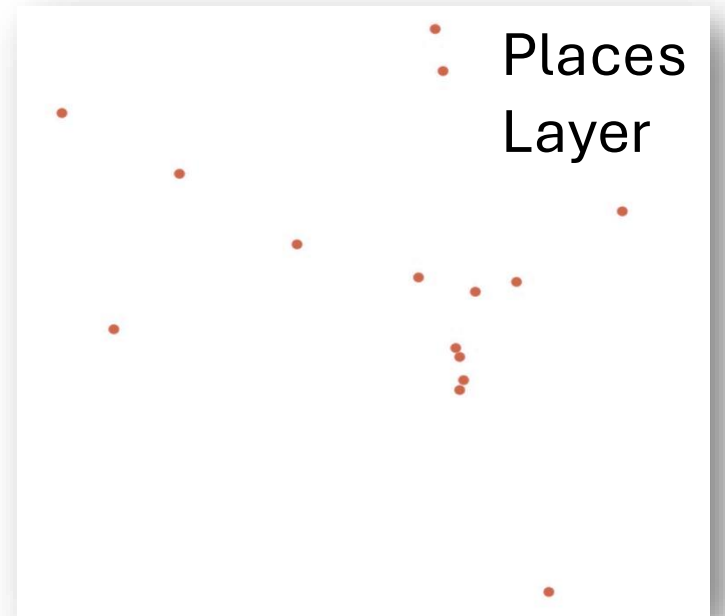
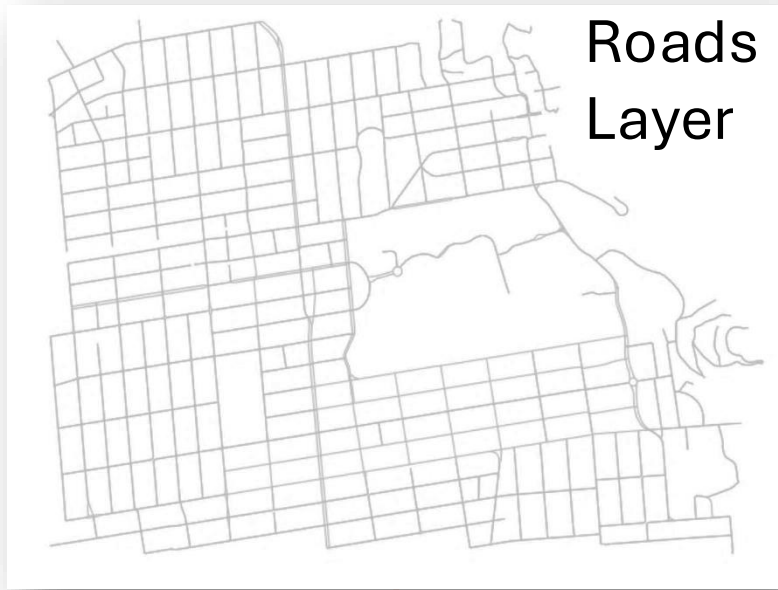
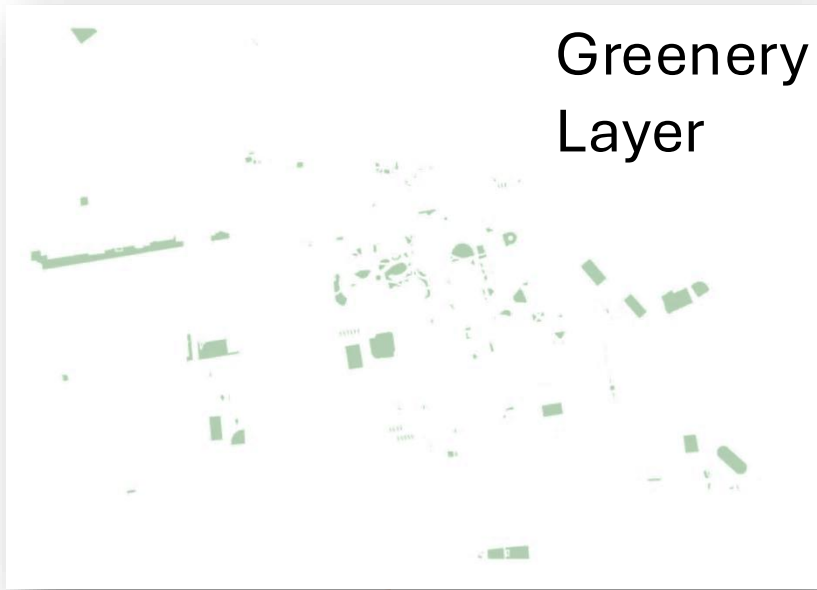
Output	Select
Spatial relationship	Distance threshold
Operation / predicate	dwithin

- **dwithin** selects features within a distance, in CRS units
- In EPSG:2227 the unit is feet, so 500 means 500 ft
- 8 street intersections are within 500 ft

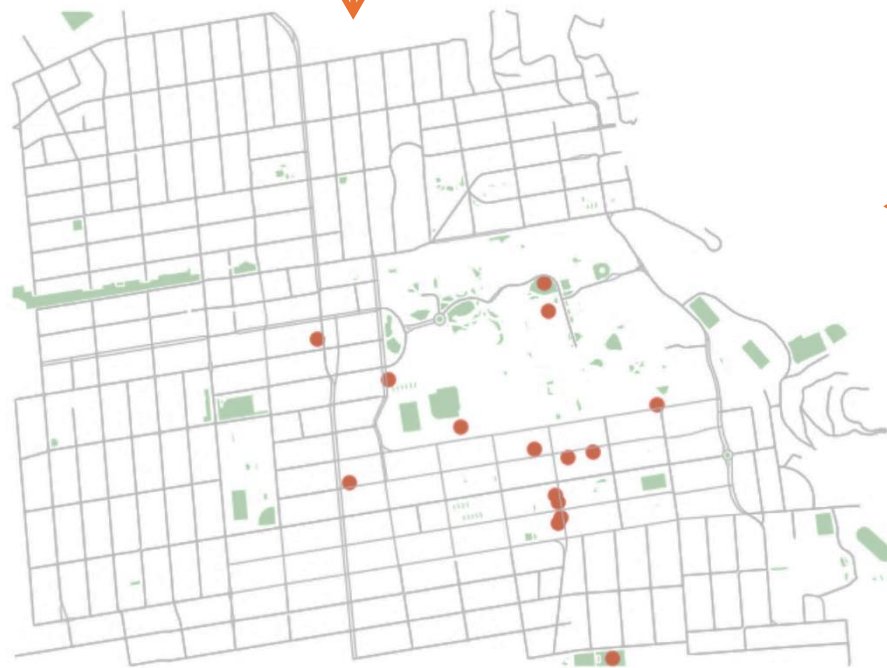
Same method as
Which intersections are within 500 ft of a crash location?

Spatial Join





Parks **intersect** Roads



Places **within** Parks

Spatial Join

- Two geometry objects are merged based on their spatial relationship to one another
- Layers need to overlap at least partially
- **ALWAYS** check the CRS, we must have **the same crs** to perform spatial join
- Spatial Join (sjoin) is between two layers at a time.
- Spatial predicates control how the spatial relationship between the geometries in the two data layers is checked

Spatial Join

```
left_gdf.sjoin(right_gdf, how, predicate)
```

- **how**: decides the type of join and which geometry is retained
 - inner: intersection of index values from both, retain only the *left geometry*
 - left: index from the first (or *left_gdf*), retain only the *left geometry*
 - right: index from the second (or *right_gdf*); retain only the *right_df geometry*

predicate: how GeoPandas decides to join (or not to join) the attributes
contains, within, touches, crosses, overlaps

Spatial Join

- Spatial predicates
- Join Types

layer 1 **join** layer 2



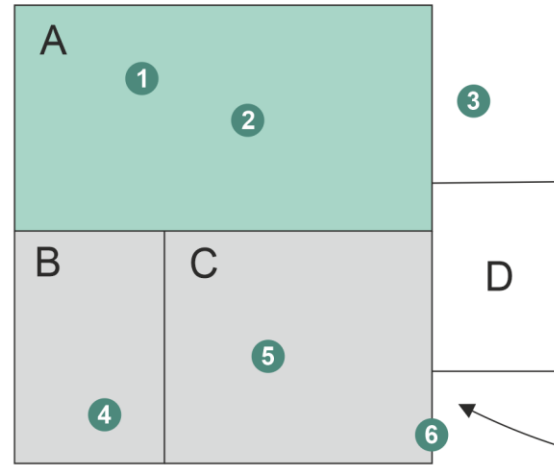
left



right

Geometric features

Layer 1: Points
Layer 2: Polygons



Attribute Tables

Layer 1

Point id
1
2
3
4
5
6

Layer 2

Name	Color
A	Green
B	Gray
C	Gray
D	White



How to join these layers based on spatial relationship?

Options

Spatial predicates

intersects
contains
within
touches
overlaps
covers
covered_by

Join types

inner join
left outer join
right outer join

Spatial join - Alternative approaches:

i

Layer 1 **within** Layer 2
-
Inner join

Point id	Name	Color
1	A	Green
2	A	Green
4	B	Gray
5	C	Gray

ii

Layer 1 **intersects** Layer 2
-
Inner join

Point id	Name	Color
1	A	Green
2	A	Green
4	B	Gray
5	C	Gray
6	C	Gray

iii

Layer 1 **within** Layer 2
-
Left outer join

Point id	Name	Color
1	A	Green
2	A	Green
3	No data	No data
4	B	Gray
5	C	Gray
6	No data	No data

iv

Layer 1 **within** Layer 2
-
Right outer join

Point id	Name	Color
1	A	Green
2	A	Green
4	B	Gray
5	C	Gray
No data	D	White

Results

Spatial Join

- Example: spatial join between point layer (e.g., taxi pickups) and a polygon layer (e.g., zip codes).

```
left_dgf.sjoin(right_gdf, how='inner', predicate='intersects')
```

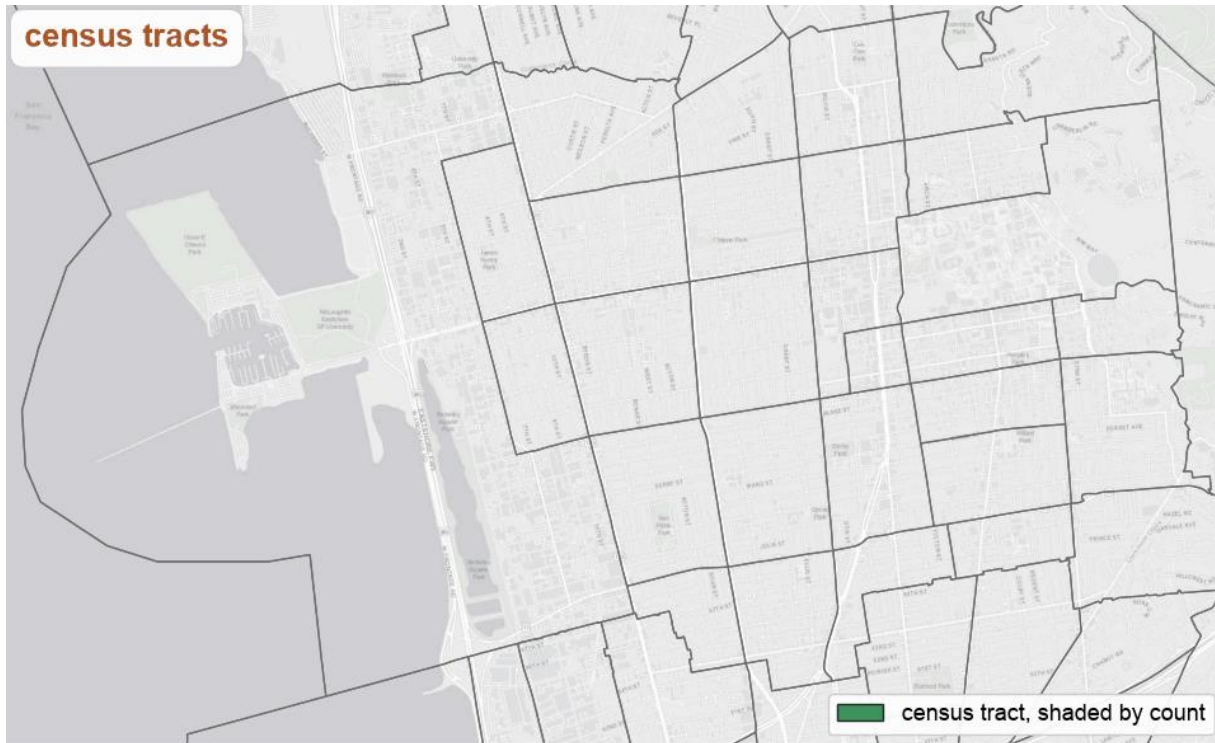


left
right



contains
within
crosses
overlaps

Containment and Count



```
in_tracts = gpd.sjoin(places, tracts, predicate="within")
in_tracts.groupby("NAME").size()
```

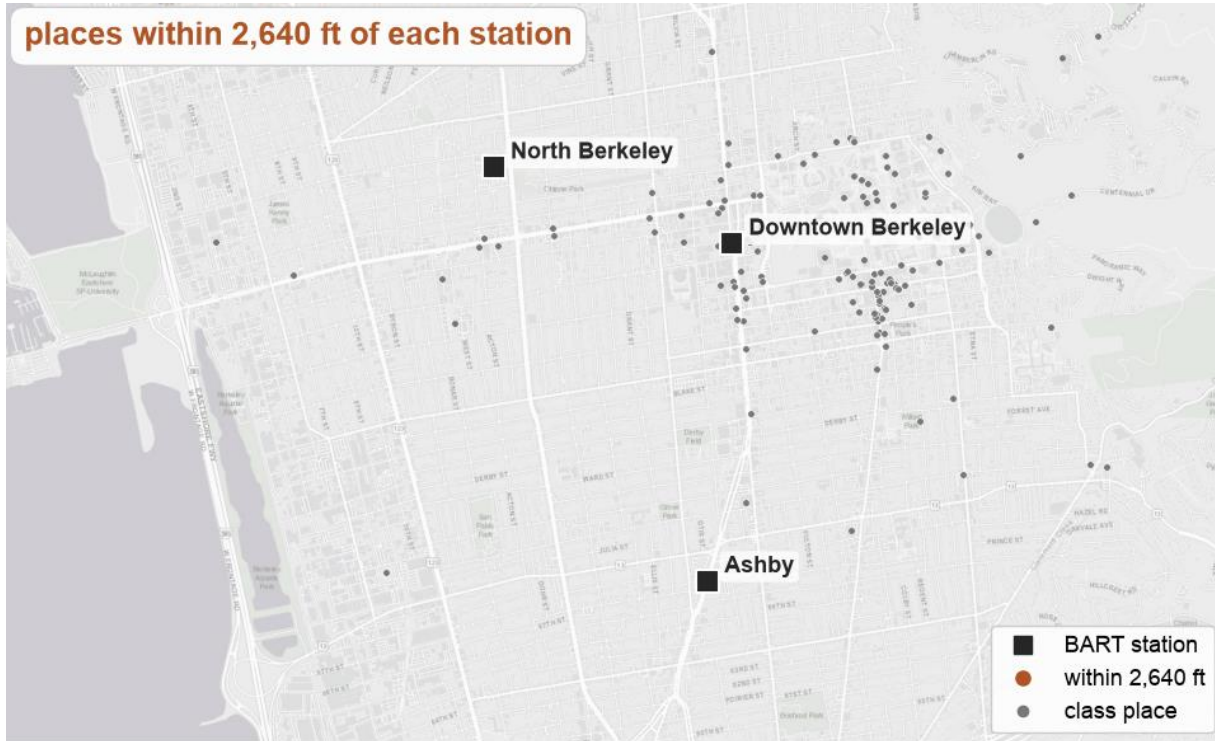
How many of the class's places are inside each census tract?

Output	Count
Spatial relationship	Containment
Operation / predicate	within + count

- sjoin attaches the tract to each place; **groupby** counts places per tract
- All 157 places fall in 22 tracts
- Campus tract 9821 has 47; Southside tract 4228 has 42

Same method as
How many residential locations are inside high heat-risk zones?

Distance Threshold and Count



```
gpd.sjoin(stations, places, predicate="dwithin", distance=2640)
.groupby("station").size()
```

How many of the class's places are within a half mile (2,640 ft) of each BART station?

Output	Count
Spatial relationship	Distance threshold
Operation / predicate	dwithin + count

- sjoin returns one row per **station-place match**
- `.groupby("station").size()` counts the matches
- Downtown Berkeley 60, North Berkeley 6, Ashby 1

Same method as
How many POIs are within 1,000 ft of each site?

Nearest Join

- This join relies on distances. Results will be inaccurate if your geometries are in a **geographic CRS (lon, lat)**.

```
left_dgf.sjoin_nearest(  
    right_gdf,  
    how: str,  
    max_distance: float, # maximum search radius for matching geometries  
    distance_col: str # name of the column, if set, will add distances  
                        in the map unit  
)
```

Nearest Neighbor - sjoin_nearest



Which BART station is nearest to each of the class's study places?

Output	Match nearest
Spatial relationship	Nearest neighbor
Operation / predicate	sjoin_nearest

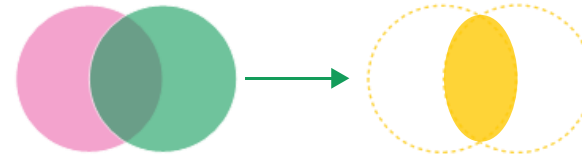
- Every study place gets **one** nearest station, however far
- `distance_col="distance_ft"` stores the distance in feet
- 32 of 34 match Downtown Berkeley; the farthest is 7,003 ft

Same method as

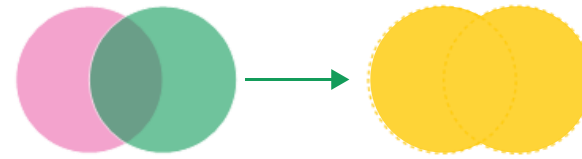
Which hospital is nearest to each senior housing building?

Spatial Overlay

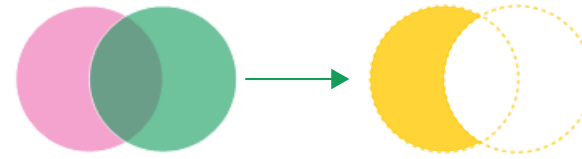
- **Intersection:** Use to find **where two areas overlap**: e.g., ZIP code regions that **fall within** a 1 km buffer around a train station. Helps identify directly affected zones.
- **Union:** Use to create a **combined area**, e.g., merge the buffer and ZIP areas into one layer to define the **total extent** of influence or service.



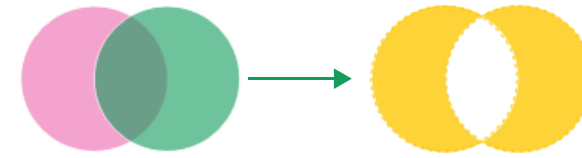
Intersection



Union



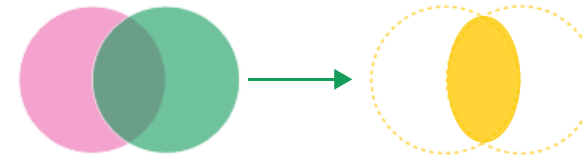
Difference



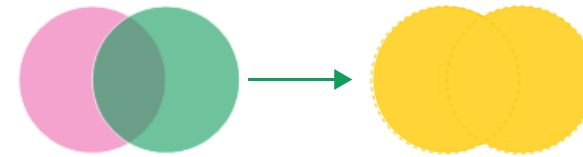
Symmetric difference

Spatial Overlay

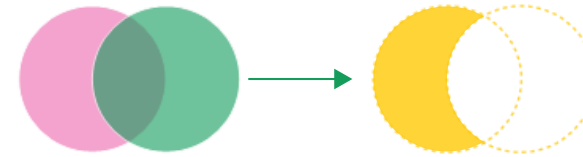
- **Symmetric difference** can be used to find **what's inside one layer but not the other**.
E.g., areas near a **train station buffer** that are **not covered** by ZIP codes, or ZIP code areas that fall **outside the service zone**. It helps identify both **coverage gaps** and **overextended boundaries** in a single step.



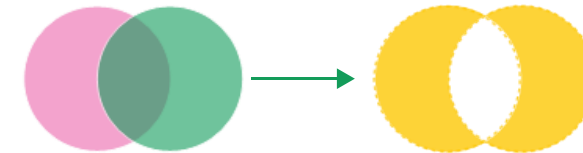
Intersection



Union

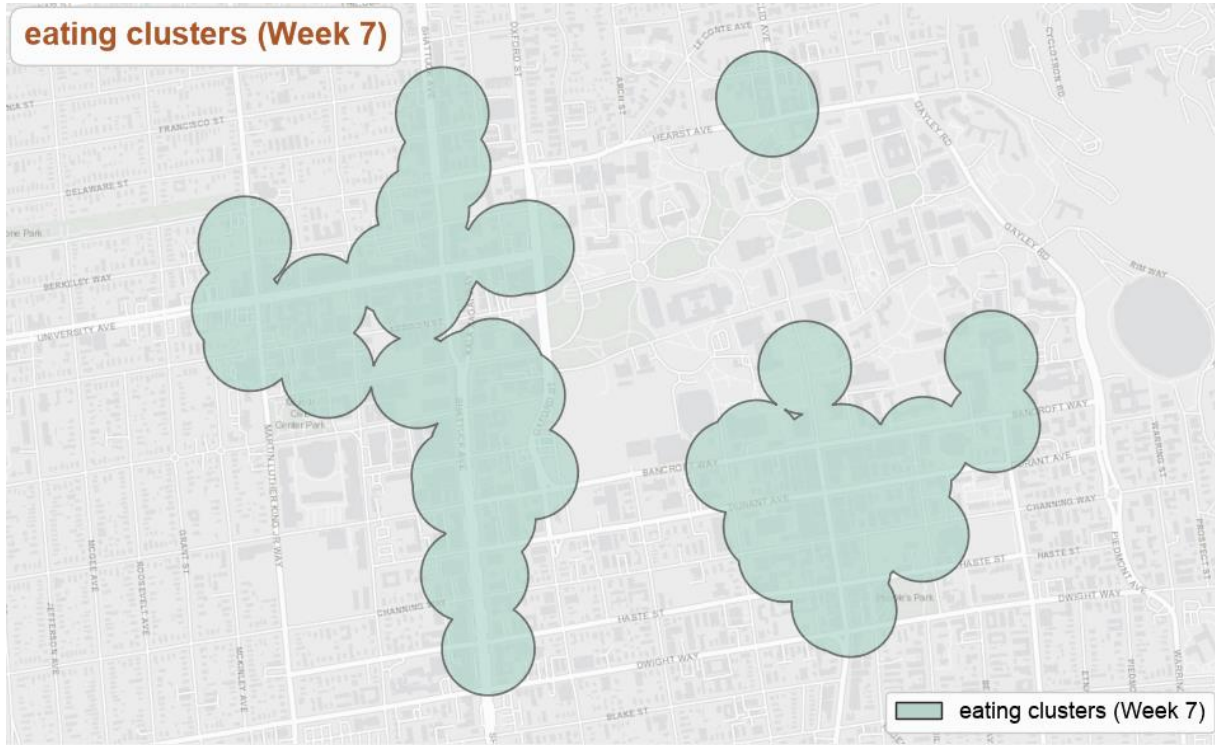


Difference



Symmetric difference

Intersection - clip



```
eat_clusters = clusters[clusters["cluster_type"] == "eat"]  
gpd.clip(streets, eat_clusters)
```

Which parts of the streets lie inside the class's eating clusters?

Output	New geometry
Spatial relationship	Intersection
Operation / predicate	clip

- **clip** cuts each segment at the cluster edge and keeps the part inside
- The same 275 segments come back, cut to 13.0 of their 18.9 mi
- Columns are copied; recompute lengths and areas after clipping

Compare with

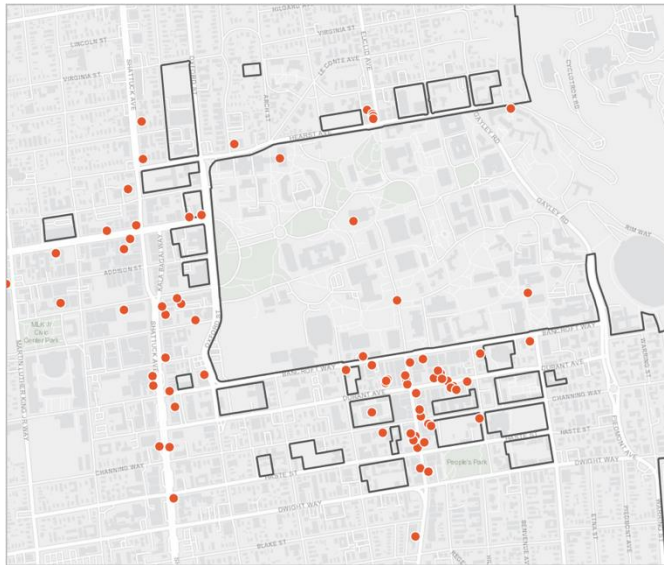
Intersection - intersects, which keeps the whole segments

Let's Practice: My Places Activity

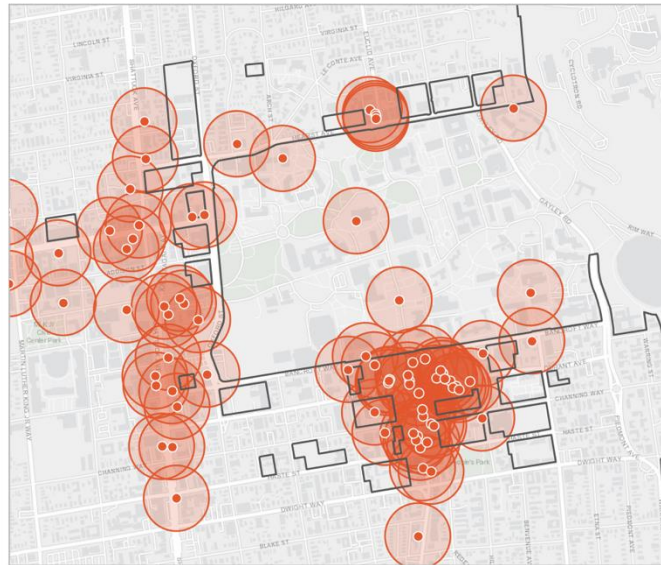
Your Places

Where does the class eat, study and unwind?

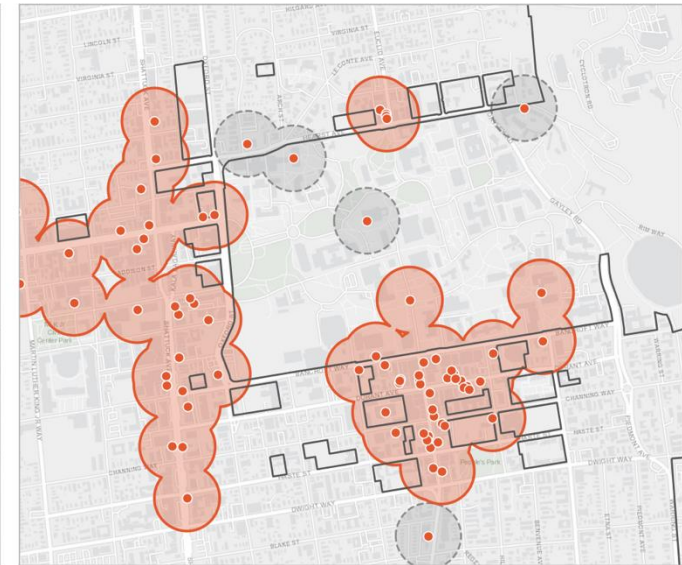
- `class_entries.csv` combines your Lab 2 files (351 entries, 75 students, 157 places)
- Convert to feet first with `.to_crs(2227)`, so `.buffer(330)` means **330 ft**



1. `places.to_crs(2227)`
eat places, one point each, in feet

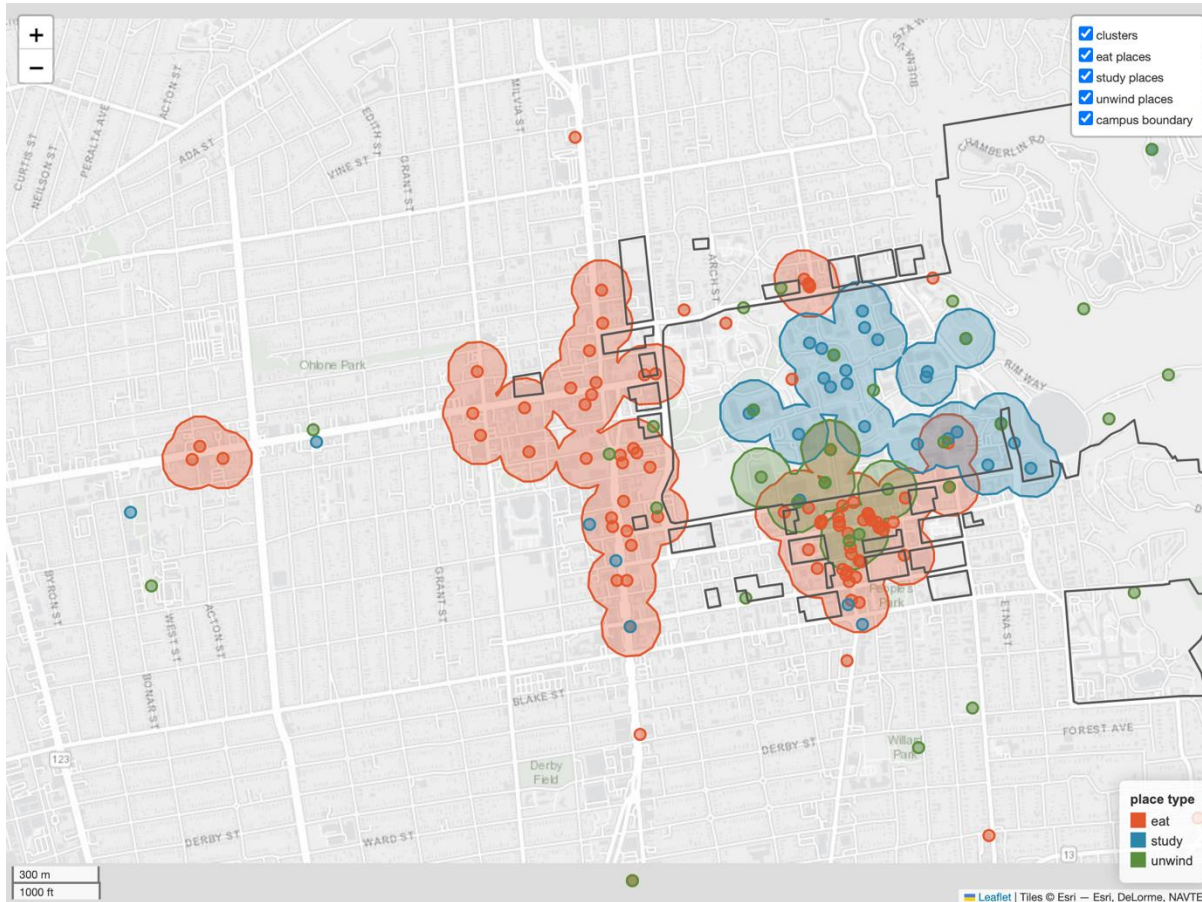


2. `.buffer(330)`
330 ft around each place



3. `.dissolve().explode()`
areas with < 3 places dropped (gray)

Your Places



- **7 clusters** (4 eat, 1 study, 2 unwind)
- Match each place to the clusters it falls in
`gpd.sjoin(places, clusters, predicate="within")`
- A row after a spatial join is a match; count places with `.nunique()`

Part B: do the same for your own places

1. Add 5 places to your Lab 2 file
2. Count your places within 1,320 ft of the place you visit most
3. Join your places to the clusters with `gpd.sjoin()`
4. Map your places on the clusters
5. Save the map as `my_places_map_w7.html` in `vis/`

2.5 bonus points for class participation

Part A by 2:10 pm (0.5), Part B by Mon Oct 12, 12 pm (2)